WHITEPAPER

A minimal Raft consensus layer on rust-libp2p

libp2p-raft — Swarm, Behaviour, Handler architecture

HoodLabs / libp2p-raft

Document version: 1.0 | November 2, 2026

This document describes a research and experimentation design.
It is not intended for production deployment.

Abstract

We present *libp2p-raft*: a research implementation that embeds a simple Raft state machine into a rust-libp2p `NetworkBehaviour`. The design separates **RaftEngine** (pure logic, synchronous, no I/O) from **RaftBehaviour** (the network adapter: maps `NodeId` ↔ `PeerId`, manages asynchronous RPCs) and **RaftHandler** (one instance per connection, unary framed RPC over the `/libp2p-raft/1.0.0` substream). The libp2p Swarm orchestrates transports, the connection pool and the event loop; behaviour and handler never call each other directly.

The paper states the architectural goals, the trust model, the wire format, the control flows (leader election, replication, inbound reply), the event-driven execution model, and the deliberate limitations compared with production Raft. It is intended as a reference for learning and experimenting with combining consensus and a modern P2P stack.

Keywords: Raft, libp2p, rust-libp2p, Swarm, NetworkBehaviour, ConnectionHandler, distributed consensus, unary RPC.

Contents

1 Introduction **3**

2 Problem and design goals **3**

2.1 Problem . . . 3

2.2 Architectural goals . . . 3

2.3 Out of scope . . . 3

3 System overview **4**

3.1 Layers . . . 4

3.2 The Swarm in libp2p . . . 4

3.3 Logical flow diagram . . . 4

4 Behaviour and Handler interfaces **4**

4.1 Associated types . . . 4

4.2 Outbound and inbound unary . . . 4

4.3 Replying on the same connection: `to == from` . . . 5

5 Wire protocol specification **5**

5.1 Substream identifier . . . 5

5.2 Framing . . . 5

5.3 In-memory decoding . . . 5

6 Consensus control flows **6**

6.1 Leader election 6

6.2 Heartbeat and replication 6

6.3 Proposing and committing commands 6

6.4 Timeouts and transport errors 6

7 Execution model **6**

8 Identity and bootstrap **6**

9 Limitations and implementation status **7**

10 Conclusion **7**

1 Introduction

A replicated state machine needs a consensus protocol so that processes agree on an ordered history of commands despite network failures or crashes. Raft [1] splits the problem into leader election, log replication and safety invariants that are easier to implement than Paxos. In parallel, libp2p [2] provides a modular networking stack: transport upgrades, cryptographic identity, substream multiplexing — applications express protocols via `NetworkBehaviour` and `ConnectionHandler` instead of raw sockets.

`libp2p-raft` explores a two-layer *composition*: a hand-written Raft engine owned by `RaftBehaviour`, which neither dials nor serializes by itself; byte framing lives in `RaftHandler`. The library does not own the `Swarm`: the host application assembles TCP, Noise and Yamux via `SwarmBuilder` and attaches `RaftBehaviour`.

2 Problem and design goals

2.1 Problem

Given a fixed set of voters (static configuration), the nodes must:

i. elect at most one leader per term;
ii. replicate the command log from the leader;
iii. commit and apply the same log prefix on a majority;
iv. communicate over an authenticated (Noise) and multiplexed (Yamux) P2P network.

2.2 Architectural goals

Design principles

- **Separate engine and network:** the engine only receives ticks, logical RPCs and proposals, and returns `Actions` — no `PeerId`, no stream handles.
- **Unary RPC over substreams:** each RPC opens a substream, exchanges one request/response, and closes it — avoiding head-of-line blocking between unrelated RPCs.
- **The Swarm is the intermediary:** behaviour ↔ handler via `FromBehaviour` / `ToBehaviour`; no direct calls.
- **Lost connection ≠ lost membership:** quorum follows the voter configuration; redials and RPC timeouts never remove a voter.

2.3 Out of scope

The implementation does **not** target production: storage may be in-memory or a simple file; AppendEntries pipelining is limited to depth 1; joint consensus and full snapshot support are incomplete. The value lies in the *layering* and the Behaviour–Handler lifecycle.

3 System overview

3.1 Layers

Table 1: Layer responsibilities

Layer	Responsibility
Application / Swarm	Build transports, `listen_on`, dial seeds, `propose`, receive `SwarmEvent::Behaviour`
`RaftBehaviour`	Engine, `PeerMap`, `pending`, `inbound_channels`, timer, `execute_actions`
`RaftHandler`	Substreams, negotiate `/libp2p-raft/1.0.0`, encode/decode frames
`RaftEngine`	`tick`, `handle_rpc`, election, replication, commit
`Storage`	Hard state, log, (snapshot)

3.2 The Swarm in libp2p

The **Swarm** is the node-level event loop: it owns transports, listeners, dialing and the connection pool (one `ConnectionHandler` per connection). The name *swarm* evokes a P2P swarm of peers — a node managing a dynamic set of connections, not one fixed client–server channel.

The application drives `swarm.select_next_some()`. The Swarm polls the behaviour and the handlers; Raft logic does not run inside the Swarm.

3.3 Logical flow diagram

```
App → propose / events
↓
Swarm ↔ RaftBehaviour ↔ RaftEngine ↔ Storage
↓ NotifyHandler / HandlerEvent
RaftHandler ↔ Yamux substream ↔ peer
```

4 Behaviour and Handler interfaces

4.1 Associated types

The `ConnectionHandler` trait requires two message types (the project defines their content):

- `FromBehaviour`: behaviour → handler — `SendRequest`, `SendResponse`.
- `ToBehaviour`: handler → behaviour — `Request`, `Response`, `Failure`.

4.2 Outbound and inbound unary

Outbound:

1. The behaviour calls `send_rpc` → `SendRequest`.

2. The handler opens an *outbound substream* on the connection to the peer.
3. It writes the frame and reads the response → `ToBehaviour::Response`.

Inbound:

1. The peer opens a substream; the handler reads the frame.
2. `ToBehaviour::Request` with a `channel_id` → the engine processes it.
3. The reply goes back via `SendResponse` on the *same* substream.

4.3 Replying on the same connection: `to == from`

When `execute_actions` is called with `Some((from, channel_id))`:

- If `Action::Send { to, msg }` has `to == from`, the behaviour uses `NotifyHandler::One(connection_id)` and `SendResponse` — no new outbound RPC is opened.
- `inbound_channels` stores `channel_id` → `{peer, conn, correlation_id}`.

`NotifyHandler::Any` is only for new outbound RPCs: the Swarm picks **one** connection to the peer. `pending` maps the `correlation_id` of RPCs **we sent** that are awaiting a response.

5 Wire protocol specification

5.1 Substream identifier

Multistream-select negotiates the protocol:

```
/libp2p-raft/1.0.0
```

5.2 Framing

Each message on a substream:

1. a 4-byte payload length, big-endian (`u32`);
2. the payload `bincode(WireEnvelope)` with `WireEnvelope { correlation_id, msg }`;
3. `msg`: a `RaftMessage` variant (RequestVote, AppendEntries, snapshot, ...).

The handler enforces a frame size limit (e.g. 4 MiB). A Raft heartbeat is an `AppendEntries` with empty `entries`.

5.3 In-memory decoding

`decode_envelope` wraps the `&[u8]` in a `std::io::Cursor` to read the length prefix through the `Read` API, then deserializes the rest — no file involved.

6 Consensus control flows

6.1 Leader election

Follower/Candidate: `election_timeout` expires → `engine.tick` → increment term, vote for self, `Broadcast RequestVote` → the behaviour opens an RPC to each voter. Majority → `BecomeLeader`, initialise `next_index` / `match_index`, start heartbeats.

6.2 Heartbeat and replication

Only the **Leader** sends `AppendEntries` every `heartbeat_interval` via `replicate_actions`. The engine emits actions → behaviour `send_rpc` → handler. Followers reset their election timeout on each heartbeat.

6.3 Proposing and committing commands

Leader: `propose(data)` persists the entry → replication → majority `match_index` → `commit_index` → `Apply` → a `Committed` event for the application.

6.4 Timeouts and transport errors

An outbound RPC past `rpc_timeout`: fail the pending entry, the engine clears `ae_inflight`; retries happen implicitly through tick/election/heartbeat. Dial failure: backoff, never treated as removing a voter.

7 Execution model

A single Swarm event loop (tokio). The engine is called **synchronously** from the behaviour — there is no separate consensus thread. The handler polls many substream I/O operations concurrently (`FuturesUnordered`); the engine remains sequential. Responses may arrive out of send order; they are matched by `correlation_id`.

The behaviour arms a `Sleep` until the earliest deadline (engine or pending RPC) and only calls `tick` when it is due.

8 Identity and bootstrap

- `NodeId` (`u64`): Raft membership, fixed in the cluster config.
- `PeerId`: libp2p identity from a keypair; the seed list maps `NodeId` ↔ `PeerId` + multiaddr.
- Voters must be configured before the first election timeout. No DHT/gossip is required for a minimal deployment.

9 Limitations and implementation status

Table 2: Compared with production Raft

Implemented	Election, replication, propose, commit events, multi-node example
Limitations	AE pipeline depth 1; reject → simple `next_index` decrement; durability depends on `Storage`; full snapshot/membership support incomplete

10 Conclusion

`libp2p-raft` illustrates a layered architecture: Raft invariants in a pure engine, rust-libp2p providing authenticated and multiplexed connections, and an adapter that uses correlation IDs, a peer map and deadline-driven ticks, and distinguishes loss of connectivity from membership change. Within the simplified scope described, the design is a compact laboratory for studying how to map a consensus state machine onto a modern P2P stack.

References

[1] D. Ongaro and J. Ousterhout, In Search of an Understandable Consensus Algorithm, in *Proc. USENIX ATC*, 2014.

[2] Protocol Labs and contributors, *rust-libp2p* — Swarm, `NetworkBehaviour`, `ConnectionHandler`.